

Quack Medicine, Good Hospitals, and Dieting

[Code ▾](#)

Foundational Concepts

Every day, we make decisions about our personal health; it's one of the most important things that we think about. Our state of health is often measured by data. The kind of data that we are interested in are things like our weight, our blood pressure, the levels of our cholesterol, and other various chemicals in our blood stream. These data present to us a picture of our well-being.

[Hide](#)

```
if (!caracas::has_sympy()) {
  caracas::install_sympy()
}
library(caracas)
library(sets)
library(tibble)
library(ggplot2)
library(dplyr)
library(sp)
library(spdep)
#library(rgeos) # Might issue a deprecation warning, but is needed for old workflow
#s
#library(geoR)
library(gstat)

# We still use the modern sf package to reliably access the data
library(sf)
library(spData)
#library(maptools) #
```

[Hide](#)

```
def_sym("X", "Y")

cmStats <- modules::use("CmStats.R")$cmStats
```

We make decisions every day about our personal health. If our cholesterol is above some number, our doctor is likely to suggest a medication to supplement healthful eating and exercise. Such a recommendation is commonly based on statistical results of studies that show a correlation between high cholesterol and increased risk of heart attack. Aspects to consider in applying results of such a study to ourselves include how like us the people in the study are and the difference between correlation and causation. Another statistical concept, regression to the mean, explains why quack medicine may appear to work often.

Condition “personalize medicine?”

What we would really like is a study involving people who are as much like us as possible, because their experience with, say, a medication is more apt to be similar to ours. We'd like to condition the overall study data on several variables, looking at the subset of the study data that matches us with respect to those variables. In general, the concept of conditioning the data on some criteria means that we look at only the data with a certain feature. We prefer to respond to the statistical results conducted on people most similar to us so that we would have a sense that the studies pertained most specifically to us.

Tailoring the statistics to our individual personal health situation could conceivably provide a greater increase in personal health than would improvements in treatments themselves. Of course, doctors are aware that different patients react differently, and to some extent, they try to tailor their treatments to the individual. Using their experience to modify the results of the studies can be good, but it can also be problematic, because an individual doctor sees many fewer patients than there are in some studies.

Another statistical issue arises when we need to decide which of two hospitals to go to for heart surgery. Suppose we have the following chart summarizing how successful each hospital is with each of three subcategories of patients: those entering in fair condition, in serious condition, and in critical condition.

Hide

```
tibble(
  patient = c("fair", "serius", "critical", "total"),
  Hospital_A = c(700, 200, 100, 1000),
  Hospital_B = c(100, 200, 700, 1000),
  Survivors_from_A = c(.600/.85, .100/.50, .10/.10, .710/.71),
  Survivors_from_B = c(.90/.90, .150/.75, .300/.43, .540/.54),
)
```

patient <chr>	Hospital_A <dbl>	Hospital_B <dbl>	Survivors_from_A <dbl>	Survivors_from_B <dbl>
fair	700	100	0.7058824	1.0000000
serius	200	200	0.2000000	0.2000000
critical	100	700	1.0000000	0.6976744
total	1000	1000	1.0000000	1.0000000
4 rows				

Looking at the data broken down in this way, we see that B has a higher success rate in all three categories of dif culties. When averaged all together, however, the impression is that hospital A is superior, with a 71% survival rate. But hospital B is superior in each of the three categories. Notice that mathematically, this is the same kind of example as one we saw in an earlier lecture on gender discrimination. This is Simpson's Paradox.

Regression to the mean

Another statistical question arises when trying to distinguish between a quack medicine and a real medicine. The reason that quack medicines sometimes seem to work is that usually people recover from a minor disease even without medicine. Quack medicines appear to work because of the phenomenon called regression to the mean: An ill person usually expects to return to his or her mean health situation.

A common issue in the realm of personal health management concerns dieting and weight. Many quantities, including weight, have an innate variability. If you weigh yourself daily and chart the results, you'll notice that the readings can differ by a pound or two, even if you didn't "gain" or "lose" weight. If you weigh yourself to the nearest half a pound, then make a histogram over the readings, you get a somewhat normal-shaped picture distributed around a central value. A weight chart is a time series, a value for different times. In the case of a person losing weight, the weight chart shows a downward trend, but it does not always go down uniformly each day. We can summarize the chart by drawing a straight line, the least squares regression line. The weight loss data illustrate how we use a mathematical model (the straight line) that captures the spirit and look of the data to summarize the data.

Several statistical issues are related to everyday health issues. We discussed the notion of conditioning on subpopulations that match a person of interest, which can give different results than analysis over the whole population. We illustrated Simpson's Paradox by the example of choosing a hospital. We illustrated how regression to the mean can be the explanation for quack medicine seeming to work and for punishment to work but praise not to work. We saw how to summarize a time series by a straight line.

Concept and Applications

SPATIAL STATISTICS

We tend to think of data as being fixed in time and space, collected in the moment to be analyzed in the moment. But what happens when our data is on the move—when values of our variables change over space? We can use a branch of statistics called spatial statistics, which extends traditional statistics to support the analysis of geographic data. It gives us techniques to analyze spatial patterns and measure spatial relationships.

SPATIAL STATISTICS

Spatial statistics is designed specifically for use with spatial data— with geographic data. These methods actually use space (area, length, direction, orientation, or some notion of how the features in a dataset interact with each other), and space is right in the statistics. That's what makes spatial statistics (also known as geostatistics) different from traditional statistical methods.

.1 Descriptive spatial statistics is similar to traditional descriptive statistics. For example, if we have lots of points on the map, we might want to know where the center of those points is located. That's equivalent to computing the mean. We might also want to know how spread out those points are around the center. That's similar to computing the standard deviation.

.2 Spatial pattern analysis tells us whether there is any (nonrandom) structure to the data. For example: Are features clustered? Are they uniform? Or are they dispersed in a regular (and nonrandom) way?

.3 Spatial analysis is used to measure and evaluate spatial relationships. Imagine that we are looking at a hot-spot map for 911 calls. We might be curious about why we are seeing so many calls, or hot spots, in certain locations. Spatial regression analysis could be used (the output looks just like regular regression, except you test for autocorrelation with a test called Moran's I). Spatial regression could identify the factors promoting the spatial pattern we're observing, which might help us explain why 911 rates are so high.

polygons. For example, customers might be points, roads could be lines, and zoning districts could be polygons. In R, any of that information can be handled in a vector format or a raster format.

Here is data for 100 counties in North Carolina that includes the counts of live births and deaths due to sudden infant death syndrome (SIDS) for 2 periods: July 1974–June 1978 and July 1979–June 1984.

Hide

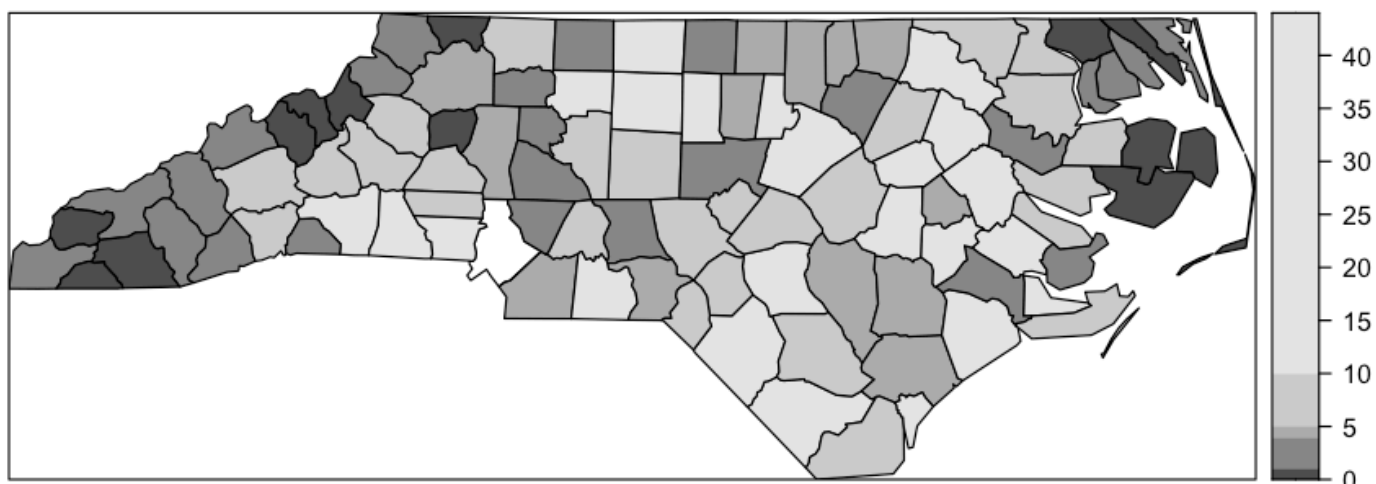
spData::nc.sids

	CNTY.ID	BIR74	SID74	NWBIR...	BIR79	SID79	NWBIR...	east	north	
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	
Ashe	1825	1091	1	10	1364	0	19	164	176	
Alleghany	1827	487	0	10	542	3	12	183	182	
Surry	1828	3188	5	208	3616	6	260	204	174	
Currituck	1831	508	1	123	830	2	145	461	182	
Northampton	1832	1421	9	1066	1606	3	1197	385	176	
Hertford	1833	1452	7	954	1838	5	1237	411	176	
Camden	1834	286	0	115	350	2	139	453	173	
Gates	1835	420	0	254	594	2	371	421	177	
Warren	1836	968	4	748	1190	2	844	344	177	
Stokes	1837	1612	1	160	2038	5	176	233	175	
1-10 of 100 rows 1-10 of 15 columns				Previous	1	2	3	4	5	6 ... 10 Next

This map has the latitude and longitude locations for the North Carolina counties along with other data from the late 1970s and early 1980s.

Who’s your neighbor? How do you decide if somebody is close to you? And how much influence should they have on your predicted value?

North Carolina SIDS Counts, 1974 (sp plot)



Hide

```
par(mar = c(0, 0, 1, 0))
plot(spdf)
invisible(text(coordinates(spdf), labels=as.character(spdf$NAME), cex=0.4))
```



Hide

```
library(spdep)

# 1. Define Queen Neighbors (Most common method)
# queen = TRUE means sharing any border or vertex is enough.
nb_queen <- poly2nb(spdf, queen = TRUE)

# 2. Define Rook Neighbors (More restrictive)
# queen = FALSE (the default) means only sharing a full border segment.
nb_rook <- poly2nb(spdf, queen = FALSE)

# Check the resulting structure (a list where each element lists the neighbor indices)
print(nb_queen)
```

```
Neighbour list object:
Number of regions: 100
Number of nonzero links: 490
Percentage nonzero weights: 4.9
Average number of links: 4.9
```

Hide

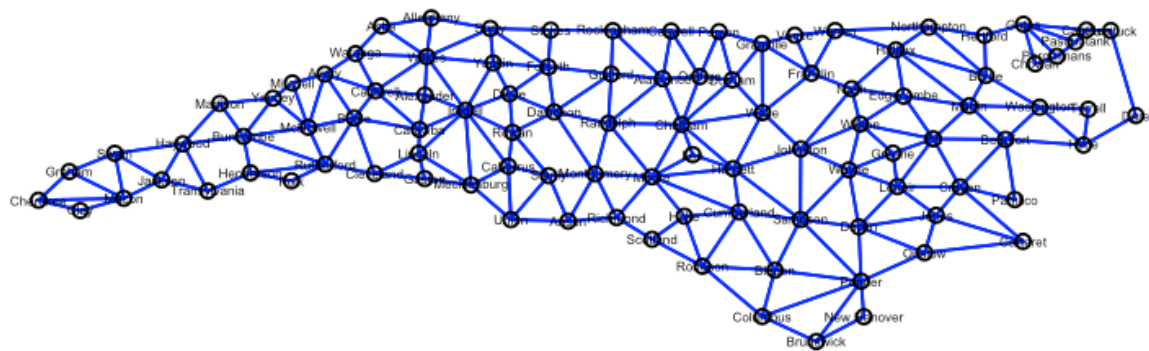
```
# Summarize the neighbor relationships
summary(nb_queen)
```

```
Neighbour list object:
Number of regions: 100
Number of nonzero links: 490
Percentage nonzero weights: 4.9
Average number of links: 4.9
Link number distribution:

 2  3  4  5  6  7  8  9
8 15 17 23 19 14  2  2
8 least connected regions:
4 21 45 56 77 80 90 99 with 2 links
2 most connected regions:
39 67 with 9 links
```

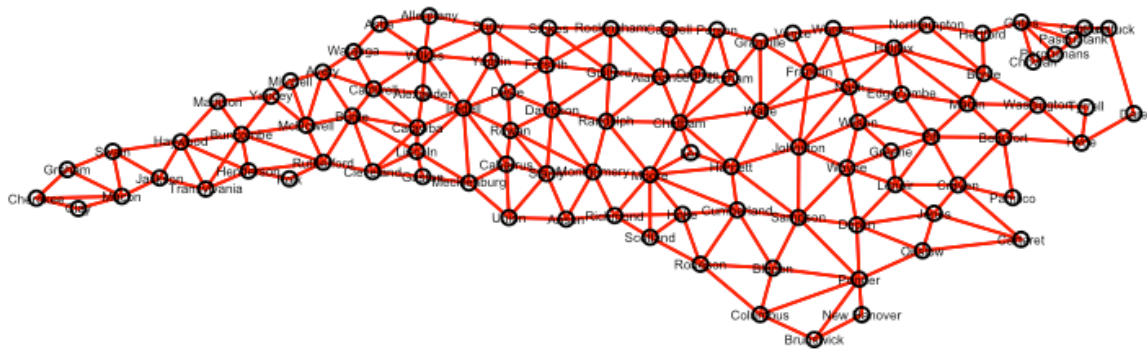
Hide

```
plot(nb_rook,
      coordinates(spdf), # Provide the coordinates (centroids) for drawing the links
      col = "blue",      # Color the links red
      lwd = 1.5           # Set line width
)
invisible(text(coordinates(spdf), labels=as.character(spdf$NAME), cex=0.4))
```



Hide

```
plot(nb_queen,
     coordinates(spdf), # Provide the coordinates (centroids) for drawing the links
     col = "red",       # Color the links red
     lwd = 1.5          # Set line width
)
invisible(text(coordinates(spdf), labels=as.character(spdf$NAME), cex=0.4))
```



Hide

```
library(spdep)

# 1. Get the coordinates (centroids) of the polygons
county_centers <- coordinates(spdf)

# 2. Find the K-Nearest Neighbors (K=3 in this case)
# The knearneigh function finds the indices of the K closest points for each point.
knn_nb_list <- knearneigh(county_centers, k = 3)

# 3. Convert the KNN list to a standard 'nb' neighbor list
# This is required for further analysis (like Moran's I) or plotting.
nb_knn <- knn2nb(knn_nb_list)

# Check the structure: it lists the indices of the 3 closest neighbors for each county.
print(nb_knn)
```

```
Neighbour list object:
Number of regions: 100
Number of nonzero links: 300
Percentage nonzero weights: 3
Average number of links: 3
Non-symmetric neighbours list
```

Hide


```
# Summarize the neighbor relationships
summary(nb_knn)
```

Neighbour list object:

Number of regions: 100

Number of nonzero links: 300

Percentage nonzero weights: 3

Average number of links: 3

Non-symmetric neighbours list

Link number distribution:

3

100

100 least connected regions:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59
60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87
88 89 90 91 92 93 94 95 96 97 98 99 100 with 3 links

100 most connected regions:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59
60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87
88 89 90 91 92 93 94 95 96 97 98 99 100 with 3 links

Hide

```
# Plot the county boundaries
```

```
plot(spdf, border = "gray", main = "North Carolina 3-Nearest Neighbors")
```

North Carolina 3-Nearest Neighbors



```
# Overlay the KNN links (lines connect the 3 closest counties)
plot(nb_knn,
     county_centers,
     col = "blue",      # Use a different color to distinguish from Queen
     lwd = 1.5
)
```

